

Roll No: C014	Name: N Mehul
Branch: B.TECH(3 RD YEAR)-C.E	Batch: 2023-2027
Date of Experiment:	Date of Submission:
Grade:	Faculty: NIKITHA PANDE

AIM:

- Apply Harris corner detector to detect corners in an image.
- Apply the techniques on different images
- Change the parameters to observe the effect on output image.
- Comment on the results before and after changing the parameters

Algorithm: Harris Corner Detection in OpenCV

Step 1: Import Required Libraries

Step 2: Read the Input Image

Step 3: Convert Image to Grayscale

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

Step 4: Convert to Float32 (Required for Harris)

```
gray = np.float32(gray)
```

Harris function needs float32 grayscale input.

Step 5: Apply Harris Corner Detection

```
cv2.cornerHarris(src, blockSize, ksize, k)
```

src → Grayscale float32 image.

blockSize → Neighborhood size considered for corner detection.

ksize → Aperture parameter of Sobel derivative (odd integer).

k → Harris detector free parameter (0.04–0.06 recommended).

Step 6: Dilate the Result for Better Visibility (Optional)

```
cv2.dilate(src, kernel)
```

kernel → Structuring element (None uses default).

Step 7: Mark Detected Corners on Original Image

```
img[dst > 0.01 * dst.max()] = [0, 0, 255]
```

Marks corners in red.

Step 8: Display the Output

CODE:

```
from google.colab import files

import cv2

import matplotlib.pyplot as plt

import numpy as np


def harris_experiment(img, blockSize, ksize, k):

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    gray = np.float32(gray)

    dst = cv2.cornerHarris(gray, blockSize, ksize, k)

    dst = cv2.dilate(dst, None)

    result = img.copy()

    result[dst > 0.01 * dst.max()] = [0, 0, 255]
```

```
return result
```

```
print("Upload TWO images for Harris Corner Detection")
```

```
uploaded = files.upload()
```

```
image_names = list(uploaded.keys())
```

```
if len(image_names) < 2:
```

```
    print("Please upload at least TWO images!")
```

```
else:
```

```
    images = [cv2.imread(name) for name in image_names[:2]]
```

```
    print("\n---- Experiment with Default Parameters ----")
```

```
    fig_default, axes_default = plt.subplots(1, 2, figsize=(10, 5))
```

```
    for i, img in enumerate(images):
```

```
        processed_img = harris_experiment(img, 4, 3, 0.05)
```

```
        axes_default[i].imshow(cv2.cvtColor(processed_img, cv2.COLOR_BGR2RGB))
```

```
        axes_default[i].set_title(f'Image {i+1} - Default Params')
```

```
        axes_default[i].axis('off')
```

```
    plt.tight_layout()
```

```
    plt.show()
```

```
    print("\n---- Experiment by Changing blockSize ----")
```

```
    fig_blockSize, axes_blockSize = plt.subplots(1, 2, figsize=(10, 5))
```

```
    for i, img in enumerate(images):
```

```
processed_img = harris_experiment(img, 8, 3, 0.05)

axes_blockSize[i].imshow(cv2.cvtColor(processed_img, cv2.COLOR_BGR2RGB))

axes_blockSize[i].set_title(f'Image {i+1} - blockSize=8")

axes_blockSize[i].axis('off')

plt.tight_layout()

plt.show()
```

```
print("\n---- Experiment by Changing k value ----")
```

```
fig_k, axes_k = plt.subplots(1, 2, figsize=(10, 5))
```

```
for i, img in enumerate(images):
```

```
    processed_img = harris_experiment(img, 4, 3, 0.1)
```

```
    axes_k[i].imshow(cv2.cvtColor(processed_img, cv2.COLOR_BGR2RGB))
```

```
    axes_k[i].set_title(f'Image {i+1} - k=0.1")
```

```
    axes_k[i].axis('off')
```

```
plt.tight_layout()
```

```
plt.show()
```

```
print("\n---- Experiment by Changing ksize ----")
```

```
fig_ksize, axes_ksize = plt.subplots(1, 2, figsize=(10, 5))
```

```
for i, img in enumerate(images):
```

```
    processed_img = harris_experiment(img, 4, 5, 0.05)
```

```
    axes_ksize[i].imshow(cv2.cvtColor(processed_img, cv2.COLOR_BGR2RGB))
```

```
    axes_ksize[i].set_title(f'Image {i+1} - ksize=5")
```

```
    axes_ksize[i].axis('off')
```

```
plt.tight_layout()
```

```
plt.show()
```

OUTPUT:

Image 1 - blockSize=8



Image 2 - blockSize=8

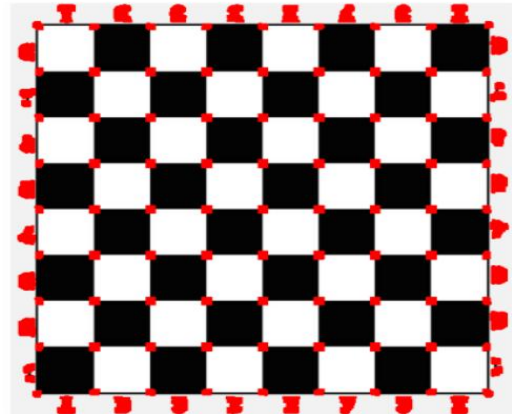
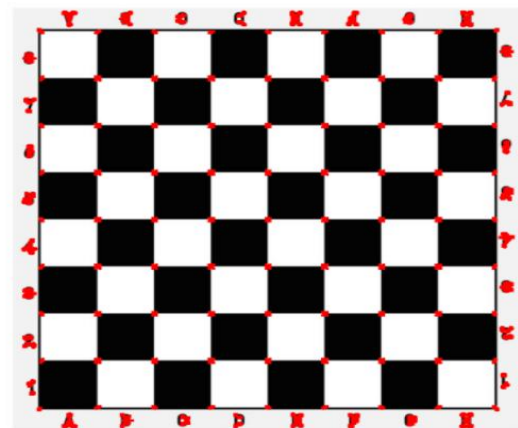


Image 1 - k=0.1



Image 2 - k=0.1



Original (Resized)



blockSize=4, ksize=5, k=0.05



OBSERVATION

1. Using default parameters, the Harris corner detector detects a large number of corners, including some weak and noisy points.
2. When the blockSize is increased, the number of detected corners reduces, and only stronger corners remain.
3. Increasing the k value makes the detector more strict, resulting in fewer but more accurate corners.
4. Increasing the ksize smoothens the image gradients, which further reduces the number of detected corners.
5. Corners are mainly detected at object boundaries, edges, and intersection points where intensity changes in multiple directions.

CONCLUSION

The Harris Corner Detection algorithm successfully detects corners in the given image. The experiment shows that parameter selection plays a crucial role in the quality of corner detection. Smaller parameter values result in more detected corners but may introduce noise, while larger values help in detecting only strong and stable corners. Thus, by properly tuning the parameters, Harris corner detection can be effectively used for feature extraction in computer vision applications.